

AWS CLOUD

PROJECT REPORT

Scalable, Secure & Highly Available Web Hosting on AWS

Services Used

EC2 • S3 • IAM • ALB • Route 53 • ACM • EBS Snapshot • AMI • Auto Scaling •
GoDaddy Domain

Date: May 2026

1. Project Overview

This project demonstrates the deployment of a scalable, secure, and highly available static website on Amazon Web Services (AWS). It covers the full infrastructure lifecycle — from provisioning a web server on EC2, storing and syncing website content via S3, securing access with IAM, creating snapshots and custom AMIs, setting up Application Load Balancer with Auto Scaling, and finally routing traffic through a custom GoDaddy domain with SSL termination using ACM.

1.1 Objectives

- Deploy a sample website on an EC2 instance running Apache Web Server.
- Store website content in an S3 bucket and sync it to EC2.
- Apply IAM roles and policies for secure S3-EC2 communication.
- Create EBS snapshots and a custom AMI for reusable golden images.
- Configure Auto Scaling with a Launch Template for high availability.
- Set up an Application Load Balancer (ALB) to distribute traffic.
- Purchase a domain on GoDaddy and configure Route 53 as the DNS provider.
- Request and attach an SSL/TLS certificate using AWS ACM.

1.2 AWS Services Summary

AWS Service	Description / Role
Amazon EC2	Virtual server to host the Apache web server and website files.
Amazon S3	Object storage bucket to upload and store website content.
AWS IAM	Identity and Access Management for EC2-S3 permission via roles/policies.
Application Load Balancer	Distributes incoming HTTP/HTTPS traffic across EC2 instances.
Amazon Route 53	DNS service for the GoDaddy domain with public hosted zone.
AWS ACM	AWS Certificate Manager to request and manage free SSL/TLS certificates.
GoDaddy Domain	Third-party domain registrar; nameservers updated to Route 53.
EBS Snapshot	Point-in-time backup of the EC2 root volume for disaster recovery.
Custom AMI	Golden machine image created from the snapshot for Auto Scaling.
Auto Scaling Group	Automatically adjusts EC2 capacity using the custom AMI & Launch Template.
Launch Template	Versioned EC2 configuration used by Auto Scaling Group.
Target Group	Logical group of EC2 instances registered with the Load Balancer.

2. Architecture Overview

The following describes the end-to-end request flow and infrastructure topology for this project.

2.1 Traffic Flow

1. User enters the custom domain (e.g., `www.yourdomain.com`) in their browser.
2. GoDaddy domain resolves DNS via Route 53 nameservers (NS records updated in GoDaddy).
3. Route 53 public hosted zone returns the ALB DNS alias (A record via Alias).
4. The ALB receives the HTTPS request and terminates SSL using the ACM certificate.
5. ALB forwards HTTP traffic to the Target Group containing healthy EC2 instances.
6. Auto Scaling Group ensures the correct number of EC2 instances (based on custom AMI) are always running.
7. EC2 instances serve the website from Apache, with content synced from S3.

2.2 High Availability Design

- Auto Scaling Group spans multiple Availability Zones for fault tolerance.
- ALB health checks automatically route traffic only to healthy instances.
- Custom AMI ensures every new instance is pre-configured with the website.
- EBS Snapshots provide a recovery point for the root volume.
- HTTPS enforced end-to-end via ACM certificate attached to ALB listener.

3. Step-by-Step Implementation

Phase 1: EC2 Instance Setup & Apache Web Server

Step	Action	Details / Commands
1.1	Launch EC2 Instance	AWS Console → EC2 → Launch Instance. Choose Amazon Linux 2023 or Ubuntu 22.04 LTS AMI. Select instance type t2.micro (or t3.small). Create/select a Key Pair (.pem file). Configure Security Group: allow SSH (port 22) from your IP, HTTP (port 80) and HTTPS (port 443) from 0.0.0.0/0. Enable 'Auto-assign public IP'. Launch instance.
1.2	Connect via SSH	chmod 400 your-key.pem ssh -i your-key.pem ec2-user@<PUBLIC_IP> (Use 'ubuntu' instead of 'ec2-user' for Ubuntu AMIs)
1.3	Install Apache	# Amazon Linux 2023: sudo dnf update -y sudo dnf install httpd -y sudo systemctl start httpd sudo systemctl enable httpd # Ubuntu: sudo apt update -y sudo apt install apache2 -y sudo systemctl start apache2 sudo systemctl enable apache2
1.4	Verify Apache	Open a browser and navigate to http://<EC2_PUBLIC_IP>. You should see the Apache default page. Alternatively: curl http://localhost
1.5	Install AWS CLI	# Amazon Linux 2023 (pre-installed). For Ubuntu: sudo apt install awscli -y # Verify: aws --version

Phase 2: S3 Bucket — Upload Website Content

Step	Action	Details / Commands
2.1	Create S3 Bucket	AWS Console → S3 → Create Bucket. Enter a globally unique bucket name (e.g., my-website-content-2026). Select your AWS Region. Keep 'Block all public access' ENABLED (access will be via IAM role, not public). Enable Bucket Versioning (optional but recommended). Click Create Bucket.
2.2	Upload Website Files	In the S3 bucket, click 'Upload'. Add your website files: index.html, style.css, images/, scripts/, etc. Or use AWS CLI: aws s3 cp ./website/ s3://my-website-content-2026/ --recursive
2.3	Verify Upload	In the S3 console, confirm all files are visible under the bucket. Check that folder structure matches your website layout.

Phase 3: IAM Role & Policy — EC2 to S3 Access

Step	Action	Details / Commands
3.1	Create IAM Policy	AWS Console → IAM → Policies → Create Policy. Choose JSON tab and paste: { "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Action": ["s3:GetObject","s3:ListBucket"], "Resource": ["arn:aws:s3:::my-website-content-2026", "arn:aws:s3:::my-website-content-2026/*"] }] } Name: EC2-S3-WebsiteAccess
3.2	Create IAM Role	IAM → Roles → Create Role. Trusted entity: AWS Service → EC2. Attach the policy: EC2-S3-WebsiteAccess. Role name: EC2-WebServer-Role. Click Create Role.
3.3	Attach Role to EC2	EC2 Console → Instances → select your instance → Actions → Security → Modify IAM Role. Select EC2-WebServer-Role → Update IAM Role.
3.4	Sync S3 to EC2	SSH into EC2 and run: <code>sudo aws s3 sync s3://my-website-content-2026/ /var/www/html/</code> # Verify: <code>ls -la /var/www/html/</code> # Test in browser: <code>http://<EC2_PUBLIC_IP></code>

Phase 4: EBS Snapshot & Custom AMI

Step	Action	Details / Commands
4.1	Create EBS Snapshot	EC2 Console → Instances → select your configured instance → Storage tab → click the Volume ID. Right-click volume → Create Snapshot. Description: WebServer-Snapshot-v1. Add Tag: Name = WebServer-Snapshot. Click Create Snapshot and wait for status to become 'completed'.
4.2	Create AMI from Snapshot	EC2 Console → Snapshots → select the snapshot → Actions → Create Image from Snapshot. Image name: WebServer-AMI-v1. Description: Apache + Website pre-configured. Architecture: x86_64. Root device name: /dev/xvda. Click Create Image. Wait for AMI status to become 'available' (EC2 → AMIs).
4.3	Tag the AMI	Select the AMI → Tags → Add/Edit Tags. Key: Name, Value: WebServer-AMI-v1. Key: Environment, Value: Production.

□ **Note:** The custom AMI captures the entire configured state of your EC2 instance — OS, Apache installation, website content, IAM agent config — so every new instance launched from it is identical and production-ready.

Phase 5: Launch Template

Step	Action	Details / Commands
5.1	Create Launch Template	EC2 Console → Launch Templates → Create Launch Template. Name: WebServer-LT. Description: Launch template for Auto Scaling web servers. Check 'Provide guidance to help me set up a template that I can use with EC2 Auto Scaling'.
5.2	Configure AMI	Under 'Application and OS Images': click 'My AMIs' → select WebServer-AMI-v1.
5.3	Instance Type & Key Pair	Instance type: t2.micro (or t3.small). Key pair: select your existing .pem key pair.
5.4	Security Group	Network settings: select existing Security Group that allows HTTP (80), HTTPS (443), SSH (22).
5.5	IAM Role	Advanced details → IAM instance profile → select EC2-WebServer-Role.
5.6	User Data (optional)	Add User Data script to auto-sync S3 on boot: <code>#!/bin/bash aws s3 sync s3://my-website-content-2026/ /var/www/html/ systemctl restart httpd</code>
5.7	Create Template	Click Create Launch Template. Note the Launch Template ID (lt-xxxxxxxxxx) — you'll need it for Auto Scaling.

Phase 6: Target Group

Step	Action	Details / Commands
6.1	Create Target Group	EC2 Console → Target Groups → Create Target Group. Target type: Instances. Name: WebServer-TG. Protocol: HTTP. Port: 80. VPC: your default VPC.
6.2	Health Check	Health check protocol: HTTP. Health check path: / (or /index.html). Advanced: Healthy threshold: 2. Unhealthy threshold: 3. Timeout: 5 sec. Interval: 30 sec.
6.3	Register Targets	Click Next. Select your existing EC2 instance → Include as pending below → Create Target Group. (Auto Scaling Group will register future instances automatically.)

Phase 7: Application Load Balancer (ALB)

Step	Action	Details / Commands
7.1	Create ALB	EC2 Console → Load Balancers → Create Load Balancer → Application Load Balancer. Name: WebServer-ALB. Scheme: Internet-facing. IP address type: IPv4.
7.2	Network Mapping	VPC: default VPC. Mappings: select ALL available Availability Zones (minimum 2 for HA).
7.3	Security Group	Create/select a Security Group for the ALB: allow inbound HTTP (80) and HTTPS (443) from 0.0.0.0/0, ::/0.
7.4	Listener — HTTP:80	Add Listener: HTTP on port 80. Default action: Forward to WebServer-TG. (We'll add HTTPS after ACM cert is ready.)
7.5	Create ALB	Review settings → Create Load Balancer. Wait for state to become 'active'. Note the ALB DNS name (e.g., WebServer-ALB-1234567890.us-east-1.elb.amazonaws.com).

Phase 8: Auto Scaling Group

Step	Action	Details / Commands
8.1	Create Auto Scaling Group	EC2 Console → Auto Scaling Groups → Create Auto Scaling Group. Name: WebServer-ASG.
8.2	Launch Template	Select the Launch Template: WebServer-LT → Latest version. Click Next.
8.3	Network	VPC: default VPC. Availability Zones: select all AZs (same as ALB). Click Next.
8.4	Load Balancing	Attach to an existing load balancer → Choose from your load balancer target groups → select WebServer-TG. Enable Elastic Load Balancing health checks. Click Next.
8.5	Capacity	Desired capacity: 2. Minimum capacity: 1. Maximum capacity: 4.
8.6	Scaling Policy	Target tracking policy. Metric: Average CPU Utilization. Target value: 50. This scales out when CPU > 50% and scales in when CPU drops.
8.7	Notifications (optional)	Add SNS topic notification for scale-out/scale-in events to your email.
8.8	Create ASG	Review and Create Auto Scaling Group. It will automatically launch 2 EC2 instances from WebServer-AMI-v1 and register them with the Target Group.

Phase 9: GoDaddy Domain & Route 53

Step	Action	Details / Commands
9.1	Purchase Domain on GoDaddy	Go to godaddy.com → Search for your desired domain (e.g., yourwebsite.com). Complete the purchase. Access your domain under My Products → Domains.
9.2	Create Route 53 Hosted Zone	AWS Console → Route 53 → Hosted Zones → Create Hosted Zone. Domain name: yourwebsite.com (exactly as purchased). Type: Public Hosted Zone. Click Create Hosted Zone.
9.3	Note NS Records	After creation, Route 53 creates 4 NS (Name Server) records. Copy all 4 nameserver values, e.g.: ns-123.awsdns-45.com ns-456.awsdns-12.co.uk ns-789.awsdns-67.net ns-012.awsdns-89.org
9.4	Update GoDaddy Nameservers	GoDaddy Dashboard → My Products → Domains → select domain → DNS. Under Nameservers → Change → Enter my own nameservers. Replace GoDaddy's default NS with the 4 Route 53 NS values. Save. Propagation takes 1–48 hours.
9.5	Create A Record (Alias)	In Route 53 Hosted Zone → Create Record. Record name: www (or leave blank for root). Record type: A. Enable Alias toggle. Route traffic to: Alias to Application and Classic Load Balancer → select your region → select WebServer-ALB. Click Create Records.
9.6	Create root redirect (optional)	Create another A record with blank name (root domain) → Alias → same ALB. This ensures both yourwebsite.com and www.yourwebsite.com resolve to the ALB.

□ **Note:** DNS propagation can take up to 48 hours, though it is usually much faster (30–60 minutes). You can verify propagation using: `nslookup yourwebsite.com` or <https://dnschecker.org>

Phase 10: SSL/TLS Certificate with AWS ACM

Step	Action	Details / Commands
10.1	Request Certificate	AWS Console → ACM (Certificate Manager) → Request Certificate. Certificate type: Request a public certificate. Click Next.
10.2	Enter Domain Names	Add domain names: *.yourwebsite.com (wildcard — covers all subdomains) yourwebsite.com (root domain) Validation method: DNS validation (recommended). Click Request.
10.3	DNS Validation	ACM provides a CNAME record for domain validation. Click 'Create records in Route 53' (automated if your hosted zone is in the same account). ACM will auto-create the required CNAME records in Route 53. Status changes to 'Issued' within 5–30 minutes.
10.4	Add HTTPS Listener to	EC2 Console → Load Balancers → WebServer-ALB →

Step	Action	Details / Commands
	ALB	Listeners tab → Add Listener. Protocol: HTTPS. Port: 443. Default action: Forward to WebServer-TG. SSL certificate: select the ACM certificate for yourwebsite.com. Click Add.
10.5	HTTP → HTTPS Redirect	Select the existing HTTP:80 listener → Edit Rules. Change the default action from 'Forward' to 'Redirect'. Redirect to: HTTPS, port 443, 301 (Permanent). This ensures all HTTP traffic is automatically redirected to HTTPS.
10.6	Verify HTTPS	Open browser → https://www.yourwebsite.com. You should see a valid padlock (🔒) and your website loads securely over HTTPS.

4. End-to-End Verification Checklist

AWS Service	Description / Role
EC2 + Apache	Navigate to EC2 public IP → Apache default page or website loads on port 80.
S3 Sync	Website files visible at /var/www/html/ on EC2. Content matches S3 bucket.
IAM Role	Run 'aws s3 ls s3://your-bucket' from EC2 without credentials — succeeds via IAM role.
Snapshot	EBS Snapshot visible in EC2 → Snapshots with status 'completed'.
Custom AMI	AMI visible in EC2 → AMIs with status 'available'.
Launch Template	Launch Template visible with correct AMI ID and IAM role.
Target Group	Target Group shows EC2 instances with health status 'Healthy'.
ALB	ALB DNS name (http://ALB-DNS) loads the website correctly.
Auto Scaling	ASG shows desired count of running instances across multiple AZs.
Route 53	nslookup www.yourwebsite.com returns ALB IP addresses.
ACM Certificate	Certificate status is 'Issued' in ACM console.
HTTPS	https://www.yourwebsite.com loads with valid padlock (no certificate warnings).
HTTP Redirect	http://www.yourwebsite.com redirects automatically to https://

5. Common Issues & Troubleshooting

5.1 EC2 / Apache

- Website not loading: Check Security Group allows port 80/443 inbound. Check Apache is running: `sudo systemctl status httpd`
- S3 sync fails: Verify IAM role is attached to EC2. Test with: `aws sts get-caller-identity`
- Permission denied on `/var/www/html`: `sudo chown -R ec2-user:apache /var/www/html`

5.2 Load Balancer

- Target shows 'Unhealthy': Check Security Group of EC2 allows traffic from ALB's Security Group on port 80. Verify Apache is running on instances.
- ALB not routing: Verify the Listener rule forwards to the correct Target Group.

5.3 DNS / Route 53

- Domain not resolving: Verify GoDaddy nameservers match exactly the 4 Route 53 NS values. Wait for propagation (up to 48 hours).
- Alias record not working: Ensure the ALB is in the same region as selected in the Alias target.

5.4 ACM Certificate

- Certificate stuck at 'Pending validation': Check that CNAME records were created in Route 53. Verify the hosted zone matches the domain exactly.
- HTTPS shows certificate warning: Ensure the ALB Listener uses the correct ACM certificate. Check that the domain matches the certificate's CN/SAN.

6. Cost Considerations

AWS Service	Description / Role
EC2 t2.micro	Free Tier eligible (750 hrs/month for 12 months). ~\$0.0116/hr beyond free tier.
S3 Storage	Free Tier: 5 GB. ~\$0.023/GB-month beyond free tier.
Application ALB	~\$0.008/LCU-hour + \$0.0225/hour base. No free tier.
Route 53 Hosted Zone	\$0.50/month per hosted zone. DNS queries: \$0.40 per million.
ACM Certificate	FREE for public certificates used with AWS services (ALB, CloudFront).
EBS Snapshot	~\$0.05/GB-month. Incremental after first snapshot.
Auto Scaling	No additional cost. You pay for the EC2 instances launched.
GoDaddy Domain	Varies by TLD. Typical .com domain: ~\$12-20/year.
Data Transfer OUT	First 100 GB/month free. ~\$0.09/GB beyond free tier.

□ **Note:** To minimize costs during development/testing: stop EC2 instances when not in use, set Auto Scaling desired capacity to 0, and delete unused snapshots.

7. Security Best Practices

- Use IAM roles instead of access keys for EC2-to-S3 communication.
- Enable S3 Block Public Access — access only via IAM, never publicly.
- Restrict SSH (port 22) to your IP address only in the Security Group.
- ALB Security Group: allow only 80/443 from the internet.
- EC2 Security Group: allow 80 only from the ALB Security Group (not 0.0.0.0/0).
- Enable HTTPS-only by redirecting HTTP to HTTPS at the ALB listener level.
- Regularly rotate key pairs and review IAM policies for least privilege.
- Enable EBS encryption for data at rest on all volumes.

8. Conclusion

This project successfully demonstrates a production-grade, highly available web hosting architecture on AWS. By combining EC2 for compute, S3 for content storage, IAM for secure access, EBS Snapshots and custom AMIs for golden images, Auto Scaling for elasticity, ALB for traffic distribution, Route 53 for DNS management, and ACM for SSL/TLS — the architecture addresses the key pillars of the AWS Well-Architected Framework: Reliability, Security, Performance Efficiency, and Cost Optimization.

The use of a custom AMI ensures that every new EC2 instance launched by the Auto Scaling Group is pre-configured and ready to serve traffic immediately. The combination of Route 53 and ACM with a GoDaddy domain provides a professional, branded, and secure endpoint accessible globally.

This architecture can be further enhanced by adding CloudFront as a CDN layer in front of the ALB, enabling DynamoDB for session management, or integrating CodePipeline for automated deployments from a Git repository.

— End of Report —